# LEMMAGEDDON: a train-booking environment that can't be broken

Team LEMMAGEDDON (Jaiv Patel, Shrey Panchal, Sahil Gupta) · RL Hackathon, "Build an RL environment that can't be broken" · Repo: `TheJaiv/LEMMAGEDDON` · Demo task: `direct_easy`, seed 0

## 1. Task

The agent is a travel booking assistant. It receives a trip request (travellers, origin, destination, arrive-by time, class, budget including cancellation fees, minimum 30-minute changes; some tasks add a pantry car, a stop limit, or "book the cheapest / best-value valid trip") and must leave behind bookings that satisfy every rule. **Success = reward ≥ 0.95.** Any valid itinerary is accepted; there is no single answer to copy.

## 2. World, tools, state

Each seed generates 6–8 Indian cities and 11–14 one-leg trains on one day. Each train has times, a type (Rajdhani, Superfast, Express, Passenger: different speed, price, stops, pantry) and classes SL / 3A / 2A with prices and seats. Domain tools only, no code execution or file access: `search_trains(origin, destination?)`, `check_seats(train, class)`, `book(train, class, travellers)`, `cancel(booking_id)` (20% fee), `list_bookings()`, `finish()`. State persists within an episode (seats taken and returned). `reset(task, seed)` always rebuilds the same world and the same start state hash.

## 3. Verifier and reward (programmatic only)

```
reward = covered × budget_score × tidy × best_choice
```

- **covered**: share of travellers on one shared valid trip (origin → destination, changes at the same station ≥ 30 min, required class, arrives by the deadline, pantry / stop limits when asked). Travellers split across trains: only the biggest group counts.
- **budget_score**: tickets kept + cancellation fees within budget 1; up to 10% over 0.5; more 0. **tidy**: –0.1 per leftover or duplicate booking (floor 0.5). **best_choice**: for "cheapest / best value" requests the verifier searches the whole timetable for the best valid trip itself; a valid but worse trip gets 0.6.
- **The verifier never trusts the sandbox.** It reads only which bookings exist and recomputes every price, fee, time and seat count from its own copy of the timetable; more seats booked than exist scores 0. Bookings are tracked by position, not id.
- Partial credit, never a free success: extra booking 0.9 · 2 of 3 seated 0.67 · valid but not cheapest 0.6 · split group 0.45 · late train 0 · book everything 0 · nothing 0. Each score carries plain-language notes.

## 4. Task set: 10 seeded generators, 3 levels

Each generator plants one valid trip (the reference solution) and trap trains that each break exactly one rule; every seed is a new variant.

| Task | Level | What makes it hard |
|---|---|---|
| `direct_easy` (demo) | easy | a cheaper train arrives after the deadline; another lacks the class |
| `soldout_easy` | easy | the earliest train is sold out in the class; a group of 2 must not be split |
| `connection_medium` | medium | no direct train fits; change at a hub (a tighter change is a trap) |
| `budget_medium` | medium | the fast direct express is 15–40% over budget; a cheaper connection fits |
| `facilities_medium` | medium | pantry car and ≤ N stops: of four train types on the route only the Superfast fits |
| `cheapest_medium` | medium | direct and connection both valid: book the cheaper (either wins about half the time) |
| `rebook_hard` | hard | an existing booking arrives late: cancel (fee counts) and rebook via a hub |
| `fix_connection_hard` | hard | an existing connection is too tight: replace only the second train; starting over busts the budget |
| `long_trip_hard` | hard | 3 travellers, only a two-change trip works; every shortcut breaks a rule |
| `value_hard` | hard | time costs Rs X/hour: fast-and-dear vs slow-and-cheap (either wins; margin ≥ Rs 82) |

## 5. Baselines and tests

- **Gate** before every model run, through the same code path: reference ≥ 0.95, no-op ≤ 0.05; otherwise the model is never called. All 10 tasks: 1.00 / 0.00.
- `tests/check_tasks.py`: 10 tasks × 300 seeds: determinism, reference passes with no tool errors, no-op ≤ 0.05, **all 9,748 planted traps < 0.95**. Mutation-tested: breaking the verifier (deadline, budget, travel-together) makes it fail.
- `tests/check_exploits.py`: 11 exploits (lookalike Cyrillic names, double bookings, someone left out, extra bookings, wrong class, injection text as a name, fake train numbers, half a trip…), 5 honest-but-unusual bookings that must still pass, 5 forged sandbox states (prices of Rs 1, more seats than exist, unknown trains), double cancel. **42,389 attempts on 200 seeds, 0 got through.** It found one real verifier weakness (repeated booking ids could hide a duplicate), fixed.

- `tests/check_harness.py`: whole episodes through the real step loop for an honest, an idle and a **cheater agent** (`mock/cheater`: reward 0.0, 13 kinds of flag). `tests/check_parsing.py`: 18 reply shapes.

## 6. Isolation

- **harness** container: generator, reference solutions, verifier, model client; internet only for the model API.
- **sandbox** container: the booking system; internal network with no internet, non-root, all capabilities dropped; its image contains no harness code.
- The template's `run_python` tool let an agent call the sandbox's own `/reset` (reproduced); our environment has no code tool.

## 7. Harness and telemetry

- Any OpenAI-compatible model; `.env`: MODEL, API_KEY, BASE_URL, TASK_ID, SEED, MODE, APPEND_FILE. 40 replies, 10 min, 10 calls/reply, 4,000-char tool output, context trimmed to ~48K tokens.
- Malformed replies and tool timeouts (60 s) get a clear error (incl. what a tool takes) and the run continues; tool calls written as text (`<tool_call>`, code blocks, JSON after prose) are read.
- One JSONL line per reply (input, raw output, actions, results, errors, state hash, reward parts, tokens, latency, flags) + termination reason and verifier notes.
- **15 suspicious-behaviour flags** (probing tools, injection text, paths/URLs, booking floods, cancel churn, false success claims, `lost_valid_trip`…). Flags never change the reward; 27 honest GPT-Luna successes raised none.

## 8. Results

| Model | Solved | Notes |
|---|---|---|
| GPT-Luna (API) | **40 / 44** (91%) | easy 8/8, medium 15/16, hard 17/20. Misses are traps working: "cancel both and start over" busted the budget (0); finished with no trip (0); booked the first direct train when a connection was cheaper (0.6); took the fast train when the slow one was better value (0.6). |
| qwen2.5:7b (Ollama) | 0 / 14 | reached a valid trip once, kept booking and lost it (`lost_valid_trip`) |
| llama3.2:3b (Ollama) | 0 / 14 | invents arguments, tools and tool output; exposed 3 parsing gaps, all fixed |

Build from a fresh clone with an empty Docker cache: 31 s; all dependencies pinned; nothing downloaded at run time.

## 9. Known limits

Times and prices are invented. The verifier searches trips of up to 4 trains (tasks need ≤ 3). Flags are pattern-based heuristics. A slow "thinking" local model can exceed the template's 120 s per reply (recorded as a crash with the reason). Next: overnight and multi-day trips, travellers joining at different stations, difficulty that grows with the seed.